# Fundamentals of Circuits and Electronics

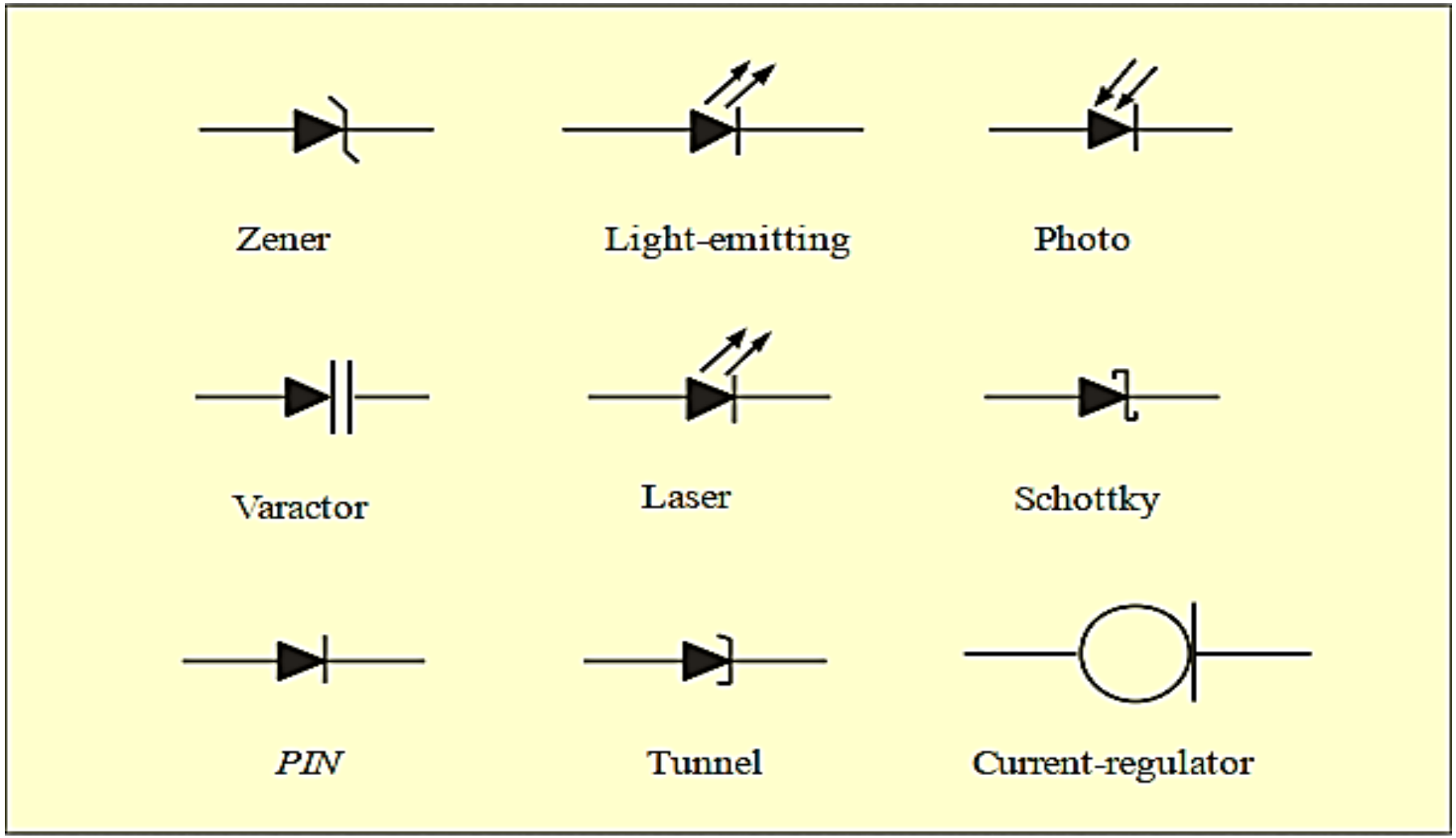
## CSAI 205

Assoc. Prof. **Mohamed Maher Ata**
Zewail city of science, technology, and innovation
momaher@zewailcity.edu.eg

# Special Purpose Diodes

- **Optical diodes:**
  - **Light-emitting diodes (LEDs)**
  - **Photodiodes**
  - Laser Diode
  - Optocouplers
- **Schottky diode**
- **Varactor diode**
- **PIN diode**
- **Tunnel diode**
- **Zener diode**

| Special purpose diode | | Methodology | Application |
|---|---|---|---|
| **Zener diode (next lecture)** | | In reverse bias, zener action introduces $V_Z$ | Voltage regulators |
| **Varactor** | | Acts like variable capacitor | TV and receiver tuners |
| **Optical diodes** | **LED** | Emits noncoherent light | DC or AC indicators |
| | **Photodiode** | Light produces minority carriers | Light detectors |
| | **Laser diode** | Emits coherent light | CD/DVD players. broadband communications |
| | **Optocoupler** | Combines LED and photodiode | Input/output isolators |
| **Schottky diode** | | Has no charge storage | High-frequency and fast switching applications |
| **PIN diode** | | Controlled resistance | Microwave communications |
| **Tunnel diode** | | Has a negative-resistance region | High-frequency oscillators |


Zener
Light-emitting
Photo
Varactor
Laser
Schottky
PIN
Tunnel
Current-regulator

# (1) Light Emitting Diodes (LED)

- In the Light Emitting Diodes (LED), if we apply a voltage across a PN junction resulting in a diode current, which in turn can produce photons and a light output of a particular color.
- The material used in the LED fabrication is called (direct-bandgap materials) such as GaAs (Gallium arsenide).
- The output light may be visible (red, blue, . . . etc) or may be invisible (Infra Red LED, GaAs).
- The wavelength λ of the emitted light is:

$$\lambda = \frac{h \times c}{E_g} = \frac{1.24}{E_g} \mu m$$

Where Eg is the bandgap energy measured in electron-volts.

- The forward voltage across an LED is typically between 1.2 V and 3.2 V, depending on the material.



Symbol of LED



# LEDs Typical Applications

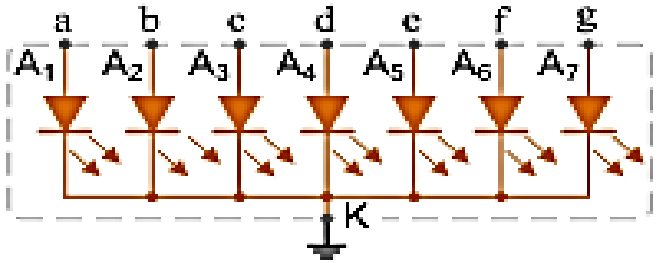
**1) TV/DVD Remote Control Unit:**

- One common application of an infrared LED is in remote control units for TV and DVD.
- The IR LED sends out a beam of invisible light that is sensed by the receiver in your TV, for example.
- For each button on the remote control unit, there is a unique code. When a specific button is pressed, a coded electrical signal is generated that goes to the LED, which converts the electrical signal to a coded infrared light signal.
- The TV receiver recognizes the code and takes appropriate action, such as changing the channel or increasing the volume.

## 2) Seven Segment display

Combinations of LEDs form light segments with the ten decimal digits. By forward-biasing selected combinations of segments, any decimal digit and a decimal point can be formed.

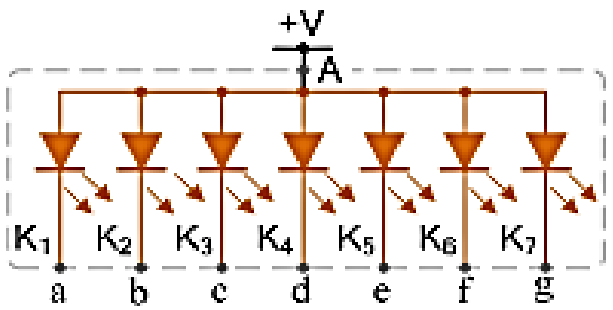
### Common Cathode (CC) Configuration




The Common Cathode (CC) Display – In the common cathode display, all the cathode (K) connections of the LED segments are tied together and connected to ground or zero-volts.

The individual segments are illuminated by the application of a suitable electric current to forward bias the individual Anode terminals (a to g). Thus common cathode display requires a driving circuit that can *source* a current.

## Common Anode (CA) Configuration




The Common Anode (CA) Display – In the common anode display, all the anode (A) connections of the LED segments are joined together to a positive voltage supply.

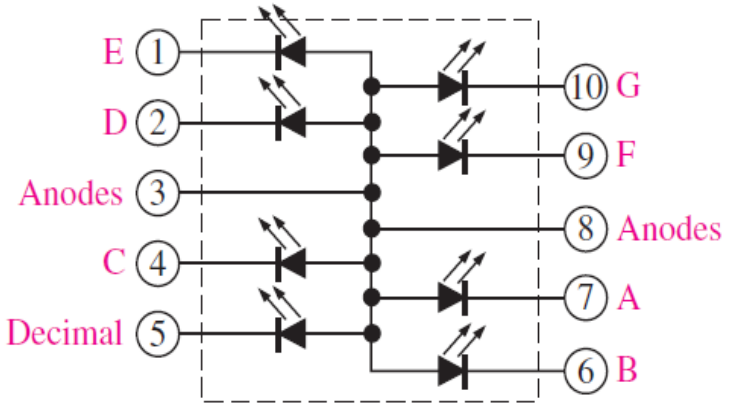
The individual segments are illuminated by applying a ground, or "LOW" signal to the Cathode terminal of the particular segment (a to g). Thus a common anode display requires a driving circuit which can *sink* a current.

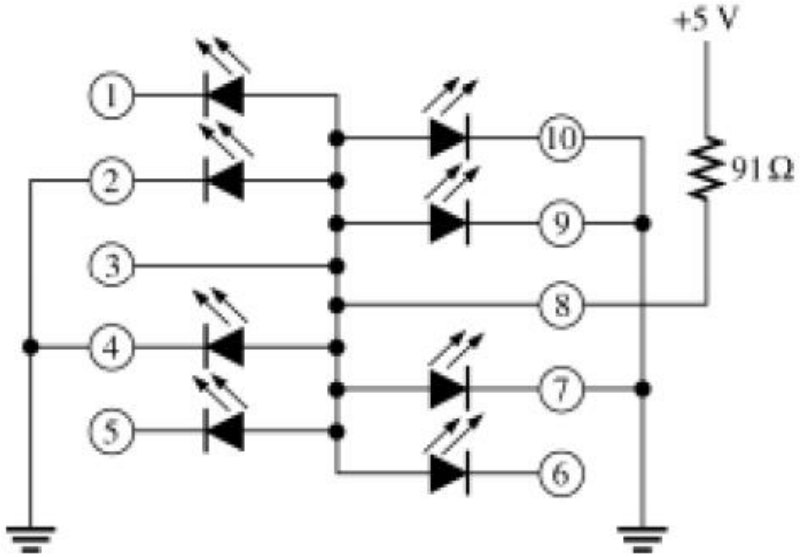
**Determine how to connect the seven-segment display in the following figure to display "5." The maximum continuous forward current for each LED is 30 mA and a +5 V dc source is to be used. Given that the forward voltage of the red LED segment is $V_F$= 2.1volts @ $I_F$=30 mA.**

$$R = \frac{V_S - V_F}{I_F}$$

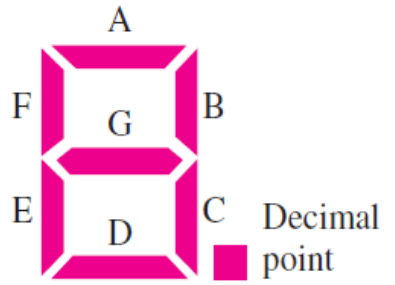
$$R = \frac{5\text{ V} - 2.1\text{ V}}{30\text{ mA}} = 97\ \Omega$$

The nearest standard 1% value is 97.6 Ω or the nearest standard 5% value is 91 Ω.




- Use common anode connection
- All anodes to Vs
- Connect cathode of (A,F,G,C,D) to ground



**Develop a yellow LED traffic-light array using a minimum number of limiting resistors that operates from a 24 V supply and consists of 100 LEDs with and an equal number of LEDs in each parallel branch. Show the circuit and the resistor values. Given that $V_F$=2.5 V @ $I_F$=30 mA.**

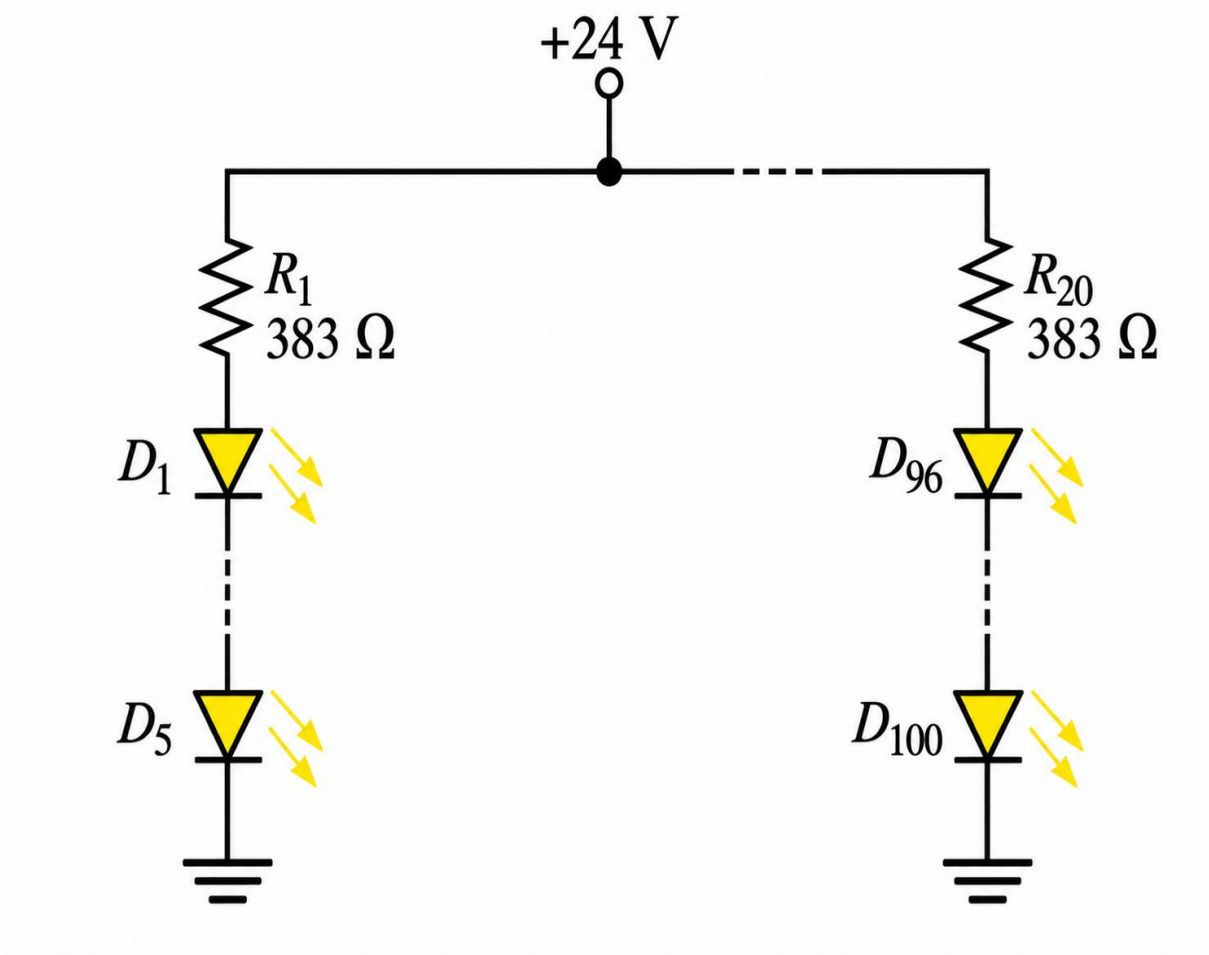
$V_F \cong 2.5$ V for $I_F = 30$ mA

$$\text{Maximum LEDs/branch} = \frac{24\text{ V}}{2.5\text{ V}} \cong 9.6$$

Select 5 LEDs/branch:

$$\text{Number of branches} = \frac{100}{5} = 20$$

$$R_{\text{LIMIT}} = \frac{24\text{ V} - 5(2.5\text{ V})}{30\text{ mA}} = 383\ \Omega$$



**Specify the number of limiting resistors and their value for a series-parallel array of 48 red LEDs using a 9 V dc source for a forward current of 20 mA. Given that $V_F$=2.2 V @ $I_F$=20 mA.**

$V_F \cong 2.2$ V for $I_F = 20$ mA

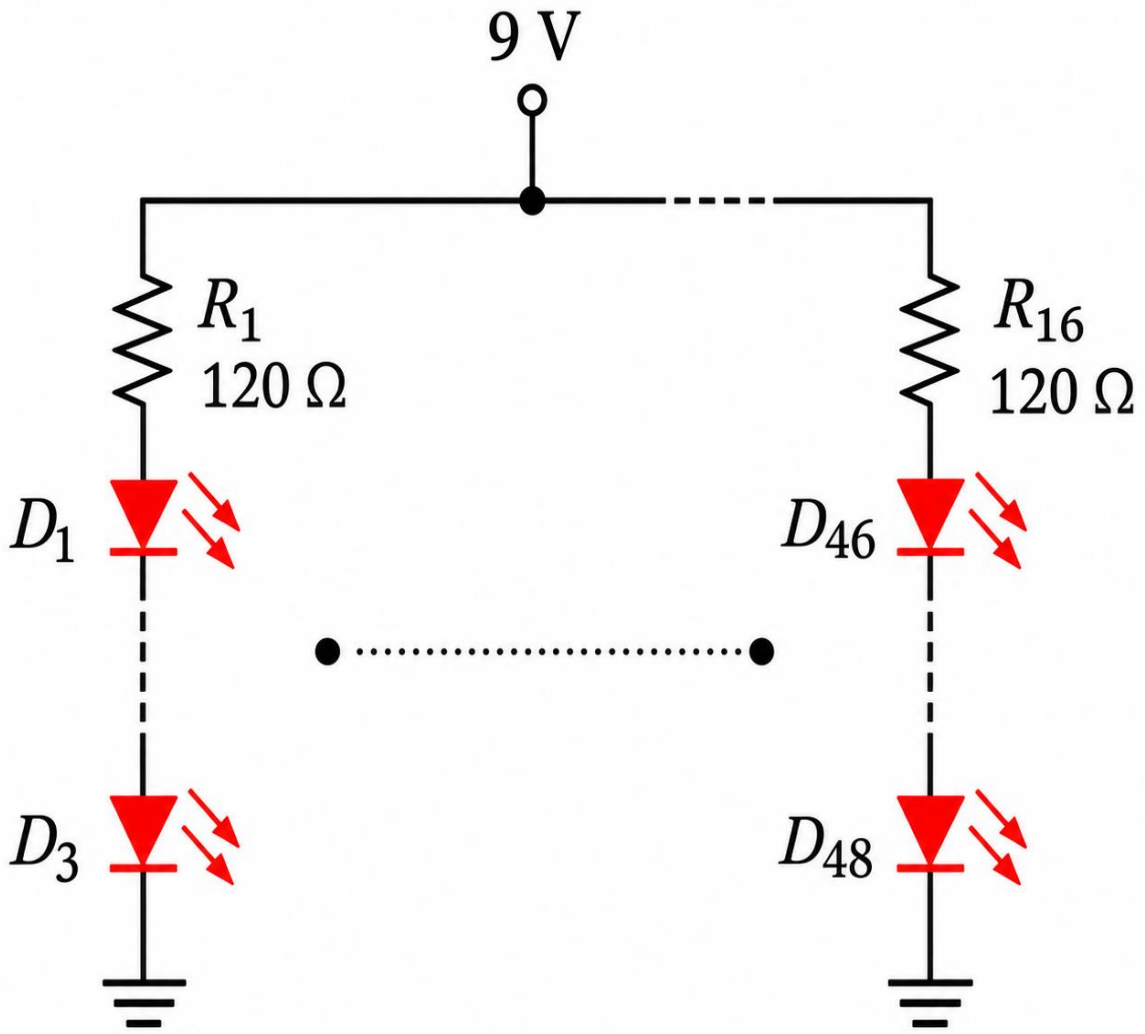
$$\text{Maximum LEDs/branch} = \frac{9\text{ V}}{2.2\text{ V}} \cong 4$$

Select 3 LEDs/branch:

$$\text{Number of branches} = \frac{48}{3} = 16$$

$$R_{\text{LIMIT}} = \frac{9\text{ V} - 3(2.2\text{ V})}{20\text{ mA}} = 120\ \Omega$$

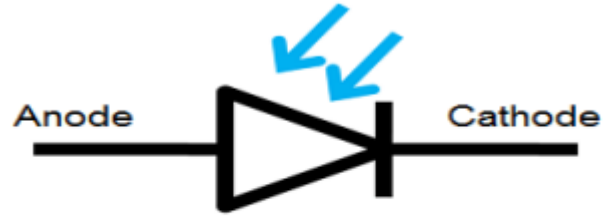
Use sixteen 120 Ω resistors.



# (2) Photodiodes

- A photodiode is a p-n junction or pin semiconductor device that **consumes** light energy **to** generate electric current. It is also sometimes referred as photo-detector, photo-sensor, or light detector
- Photodiode is very sensitive to light so when light or photons falls on the photodiode it easily converts light into electric current.
- Solar cell is also known as large area photodiode because it converts solar energy or light energy into electric energy. However, solar cell works only at bright light.

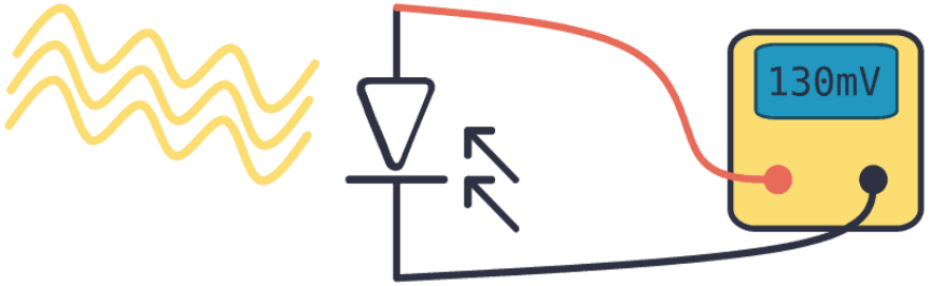





Photodiode symbol

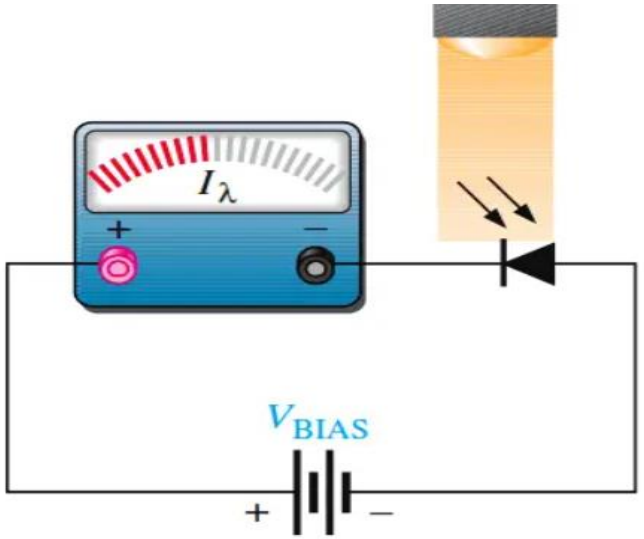
## 1. No Bias

- When you operate a photodiode without any bias voltage, shining light on it generates a small voltage across its terminals.




## 2. Reverse Bias (basic operation of photodiodes)

- When there is **no incident light**, the reverse current, I, is almost **negligible** and is called the dark current.
- When there is **incident light** and a photodiode is reverse-biased, depletion region is widened and photons penetrates the depletion region.
- When light photons hit the photodiode, their energy excites electrons, freeing them from atoms within the depletion zone. This action creates electron-hole pairs. The electric field quickly separates these charges, with electrons moving to the n-side and holes to the p-side, generating a current proportional to the light's intensity. Thus, as more light strikes the photodiode, more current flows in the circuit.




## 3. Forward Bias (not used for photodiodes)

The forward bias allows a large current to flow across the diode due to the forward voltage, making it harder to distinguish any small light-generated current. A forward-biased photodiode **is not used** for light detection as its light sensitivity is minimal in this mode.

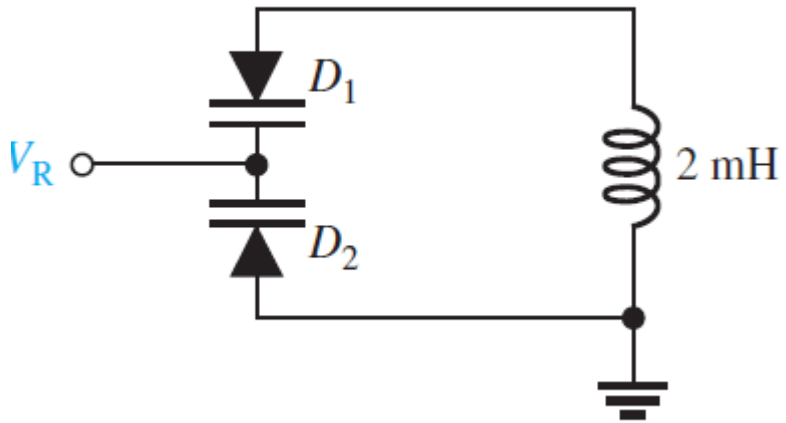
# (3) Varactor

- A **varactor**, also known as a **variable capacitor diode**, is a special type of semiconductor diode designed to act as a **voltage-controlled capacitor**. When a reverse voltage is applied to the varactor, the width of its depletion region changes, which in turn varies its capacitance. Higher reverse voltage results in a wider depletion layer and lower capacitance, while lower voltage gives higher capacitance. Varactors are commonly used in **tuning circuits**, such as radio frequency (RF) oscillators and voltage-controlled oscillators (VCOs), where they allow electronic tuning without mechanical components.

**Ex: What capacitance value is required for each of the varactors in the following figure to produce a resonant frequency of 1 MHz?**

$$f_r = \frac{1}{2\pi\sqrt{LC_T}} \rightarrow {f_r}^2 = \frac{1}{4\pi^2 LC_T}$$

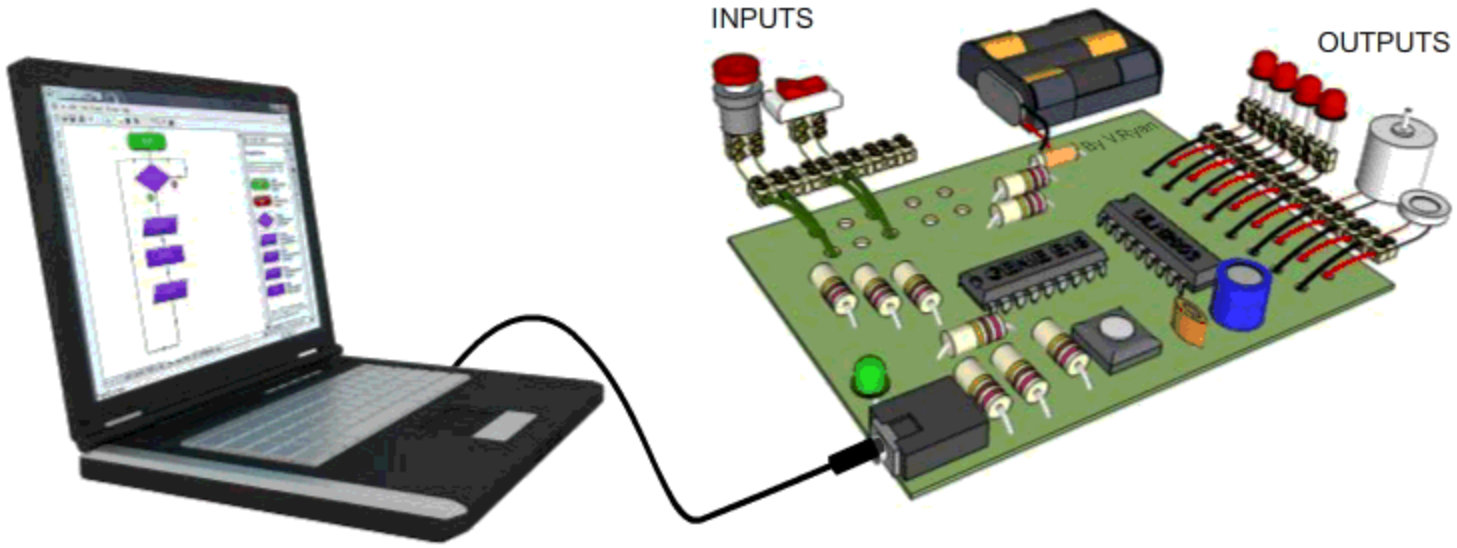
$$C_T = \frac{1}{4\pi^2 L {f_r}^2}$$




- **Note that** both Varactor are in series. Accordingly each Varactor must have capacitance of 25.4 pF

# Introduction to Embedded Systems

# What is a Microcontroller




- A small computer on a single chip
  - containing a processor, memory, and input/output
- Typically "**embedded**" inside some device that they control
- A microcontroller is often small and low cost

# Microcontroller Devices




Atmel AVR




AVR



ATX Mega



ATmega 328P




PIC 18F877A



8051



Arduino




ARM

# Control System with Arduino

Read Inputs → Process → Control Outputs

## Inputs (Sensors & Switches)



**Temperature Sensor**
Measures temperature



Analog (Voltage)



**Light Sensor (LDR)**
Measures light intensity



Analog (Voltage)



**PIR Motion Sensor**
Detects human motion



Digital (HIGH/LOW)



**Ultrasonic Sensor**
Measures distance



Digital (Pulse)



**Push Button**
User input



Digital (HIGH/LOW)



**Potentiometer**
Variable input (e.g., speed)



Analog (Voltage)



**Limit Switch**
Detects position/end point



Digital (HIGH/LOW)

Input Signals

### Controller (Arduino UNO)

- Reads input signals
- Processes data (program)
- Makes decisions
- Controls output devices




Control Signals

## Outputs (Actuators)



**LED**
Visual indicator
(e.g., status, alerts)



**Buzzer**
Sound alert
(e.g., alarm)



**DC Motor**
Rotation
(e.g., fan, wheel)



**Servo Motor**
Controlled angle
(e.g., door, camera)



**Robot Arm**
Pick and place
(e.g., object handling)



**Water Pump**
Fluid control
(e.g., irrigation, sprinkler)



**Relay**
Switch high-power devices
(e.g., lights, AC appliances)

## Example Real-World Applications

### Automatic Lighting





LDR (light level) → Arduino → LED (light)

### Security Alarm





PIR sensor (motion) → Arduino → Buzzer (alarm)

### Obstacle Avoiding Car





Ultrasonic (distance) → Arduino → DC Motor (movement)

### Robotic Arm Control





Push Button (command) → Arduino → Servo/Robot Arm (movement)

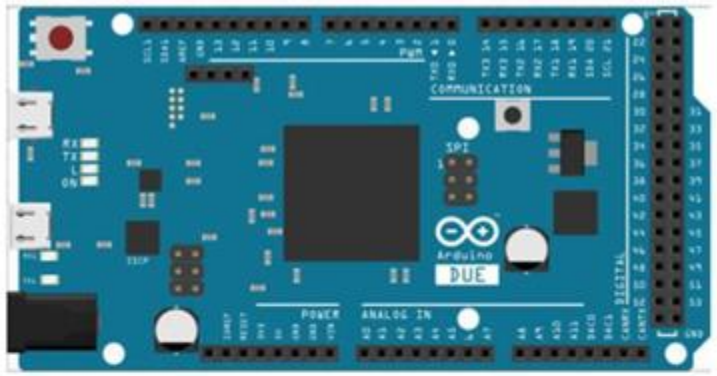
### Irrigation System



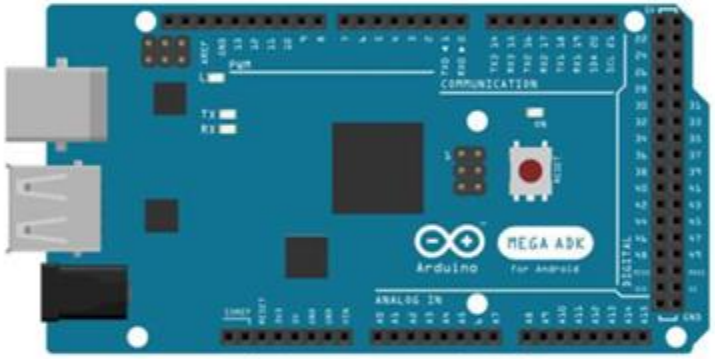


Temperature (or moisture) → Arduino → Water Pump (watering)

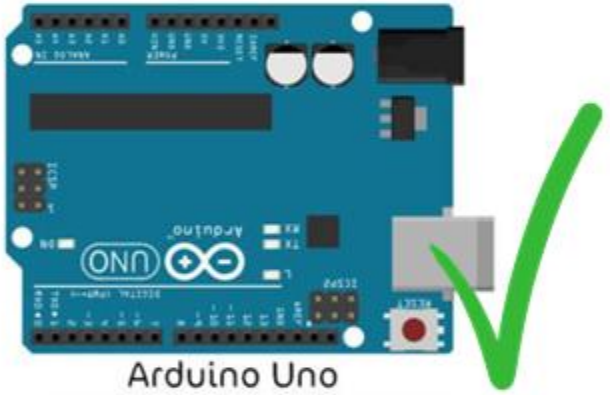
# Types of Arduino




Arduino Due




Arduino Mega 2560



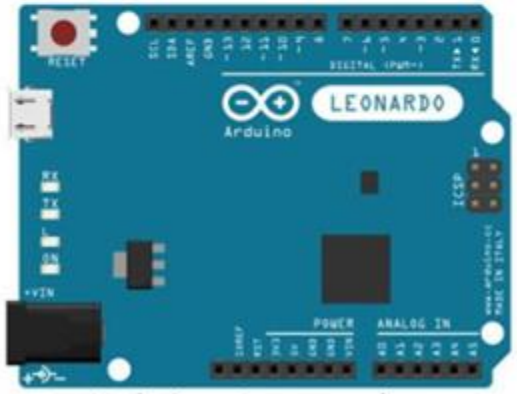

Arduino Uno

Arduino Nano



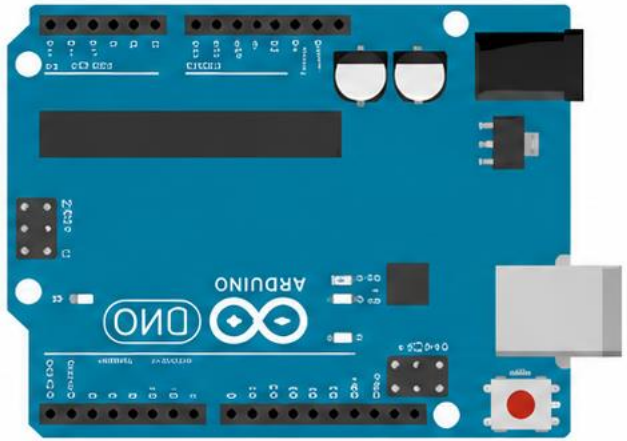


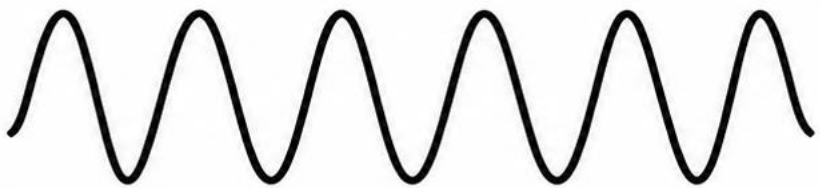
Arduino Mini



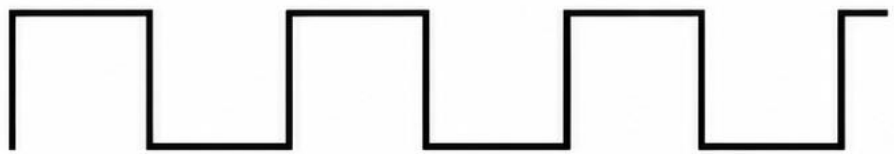

Arduino Leonardo

ATMEL



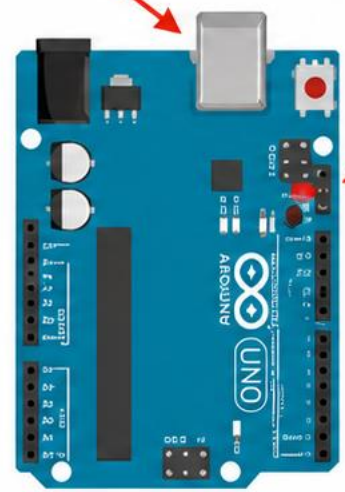


Analog Signal



Digital Signal

16 MHz
Crystal Oscillator



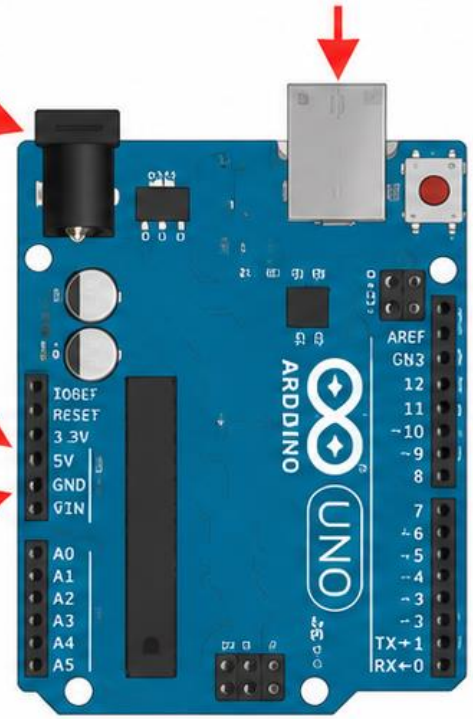
The built-in LED on pin 13 can be used as an output indicator.

**1-** The Arduino board can be powered through a USB cable.

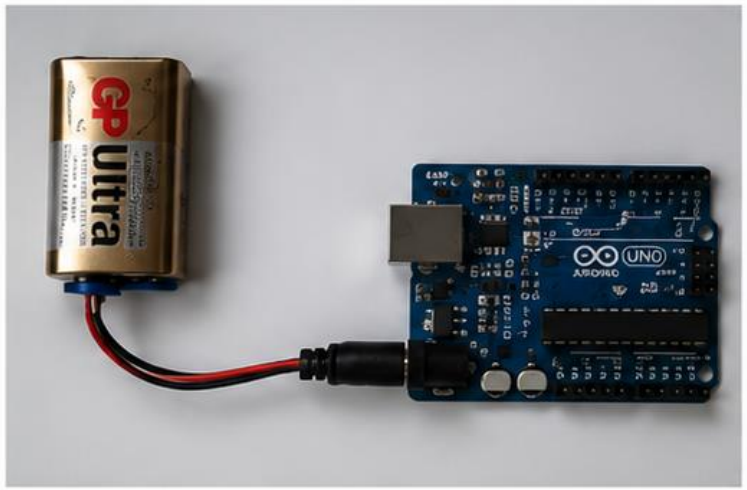
**2-** Or through a battery using the power jack (7V to 20V).

**3-** Or through the 5V pin (5V only).

**4-** Or through the VIN pin (7V to 20V).



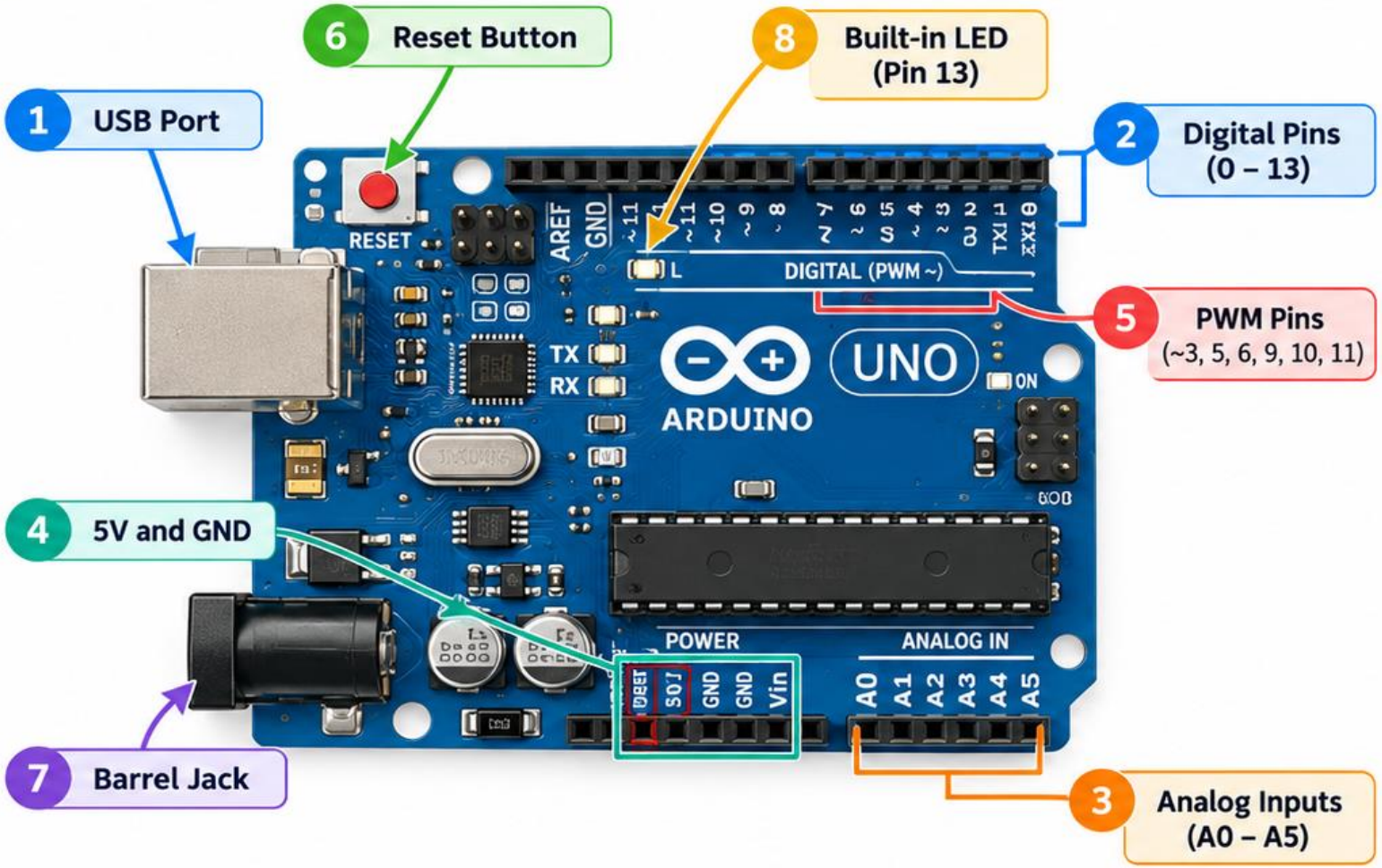





# Arduino UNO: Main Parts You Should Know

A beginner-friendly view of the board and what each part means




| Part | Meaning for students |
|---|---|
| 1 **USB Port** | Connects Arduino to the computer and uploads your program. |
| 2 **Digital Pins (0 – 13)** | Read HIGH/LOW signals or control devices ON/OFF. |
| 3 **Analog Inputs (A0 – A5)** | Read changing voltages from sensors. |
| 4 **5V and GND** | Provide power and a shared ground for circuits. |
| 5 **PWM Pins** (~3, 5, 6, 9, 10, 11) | Act like analog output using fast pulses, useful for LED brightness or motor speed. |
| 6 **Reset Button** | Restarts the uploaded program. |
| 7 **Barrel Jack** | Lets you power the board from an external adapter. |
| 8 **Built-in LED (Pin 13)** | A small onboard LED often used for simple output tests. |

**Think of Arduino as:**

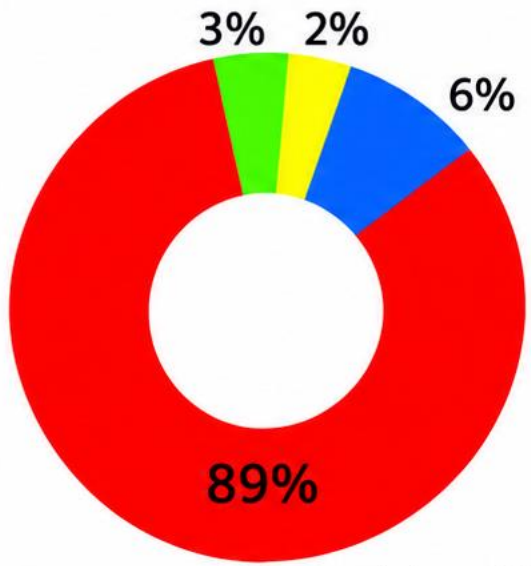
**Inputs** (e.g., sensors, buttons) + **Processing** (Arduino UNO) + **Outputs** (e.g., LEDs, motors, buzzers)

The **Boot Loader** is a small program stored in the microcontroller that is responsible for loading the user program from the computer to the microcontroller's memory. ← **Boot Loader**

**SRAM** is the memory used to store variables and data temporarily during program execution. ← **SRAM**

**Flash Memory** is the memory where the user program is stored permanently, even after power is removed. ← **Flash Memory**

**EEPROM** is a memory used to store data permanently, which can be updated during program execution and is not lost when the controller is turned off. ← **EEPROM**



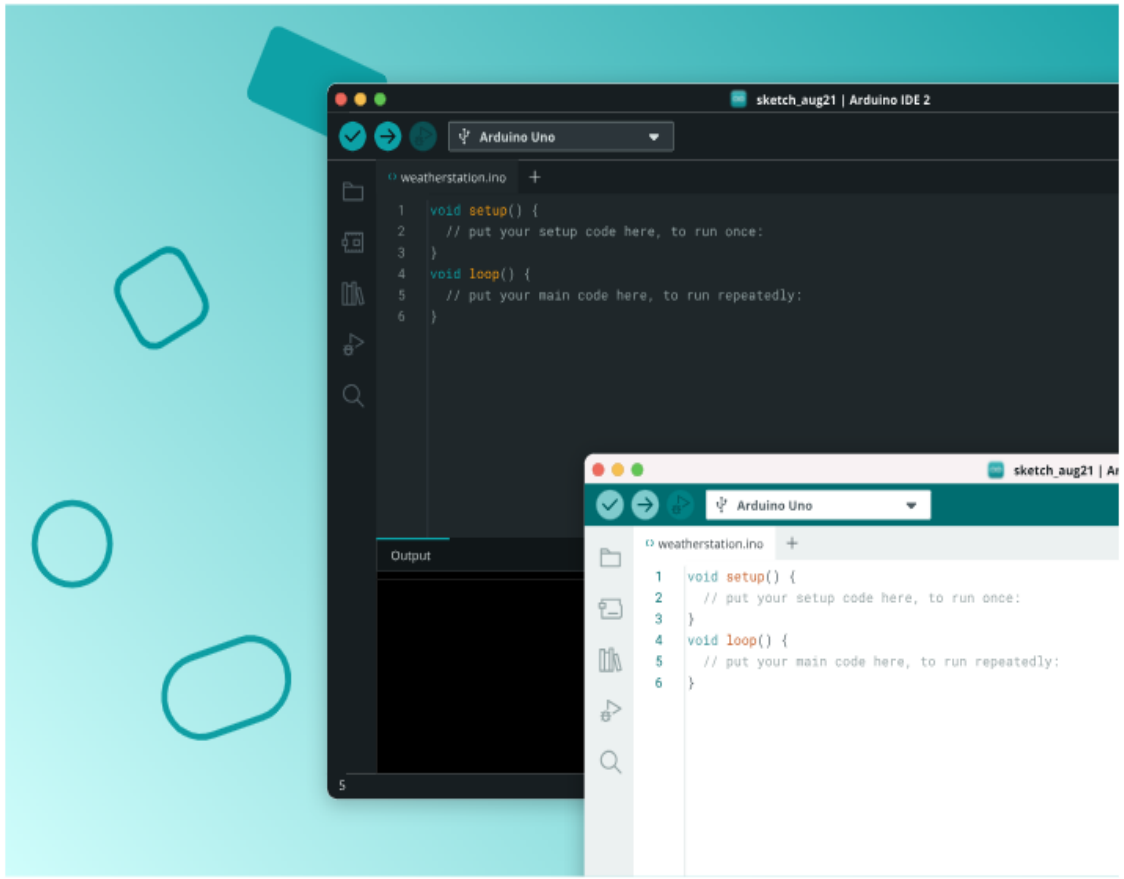

## Arduino Memory Resources:

- SRAM – 2 Kilobyte
- Flash Memory – 29 Kilobyte
- EEPROM – 1 Kilobyte
- Boot Loader – 0.5 Kilobyte

This chart is for a 32% processor running at 16 MHz.

# How to download Arduino IDE

https://www.arduino.cc/en/software/




## Arduino IDE 2.3.10

Release notes

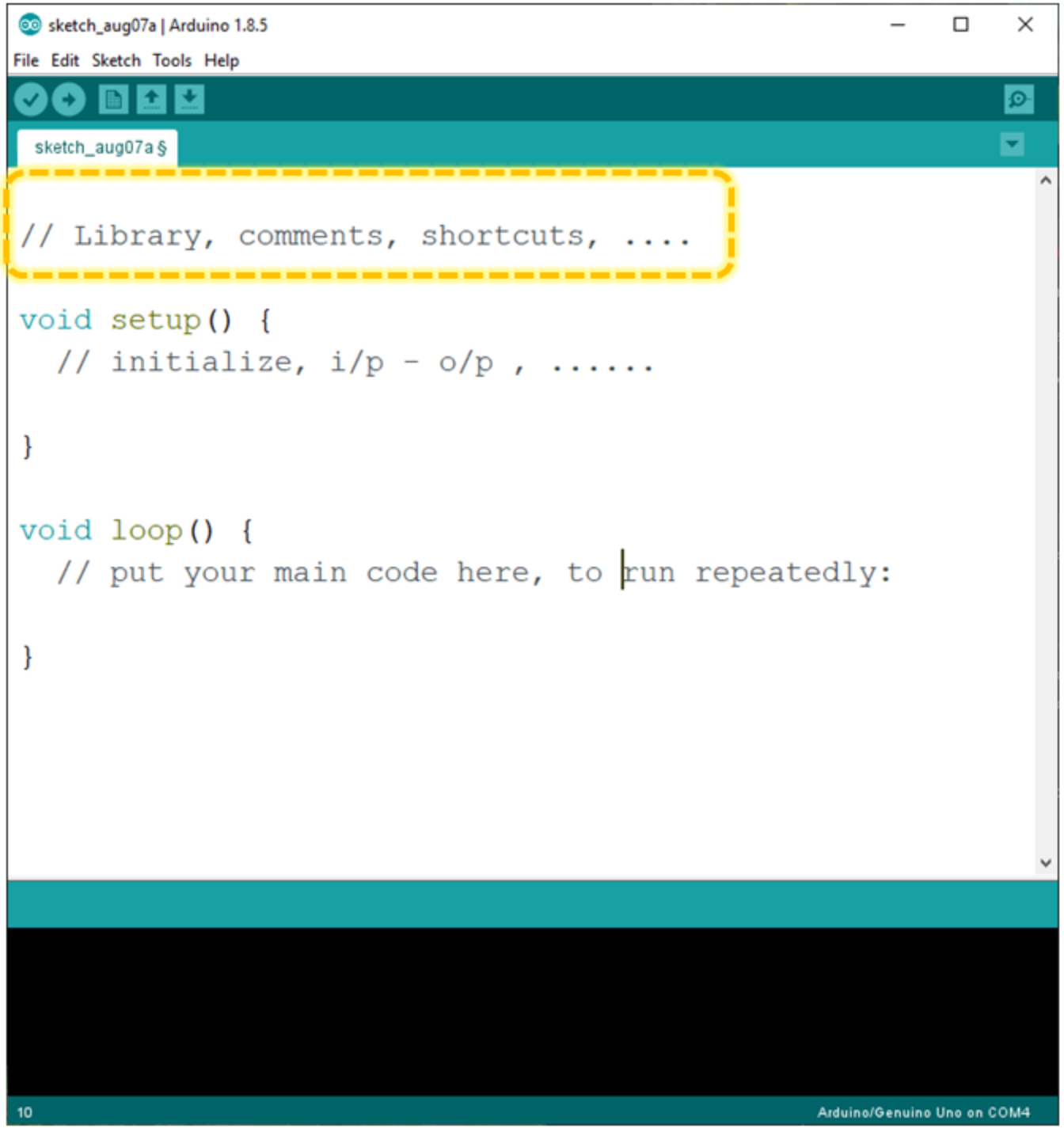
The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger. For more details, check the **Arduino IDE 2.0 documentation**.

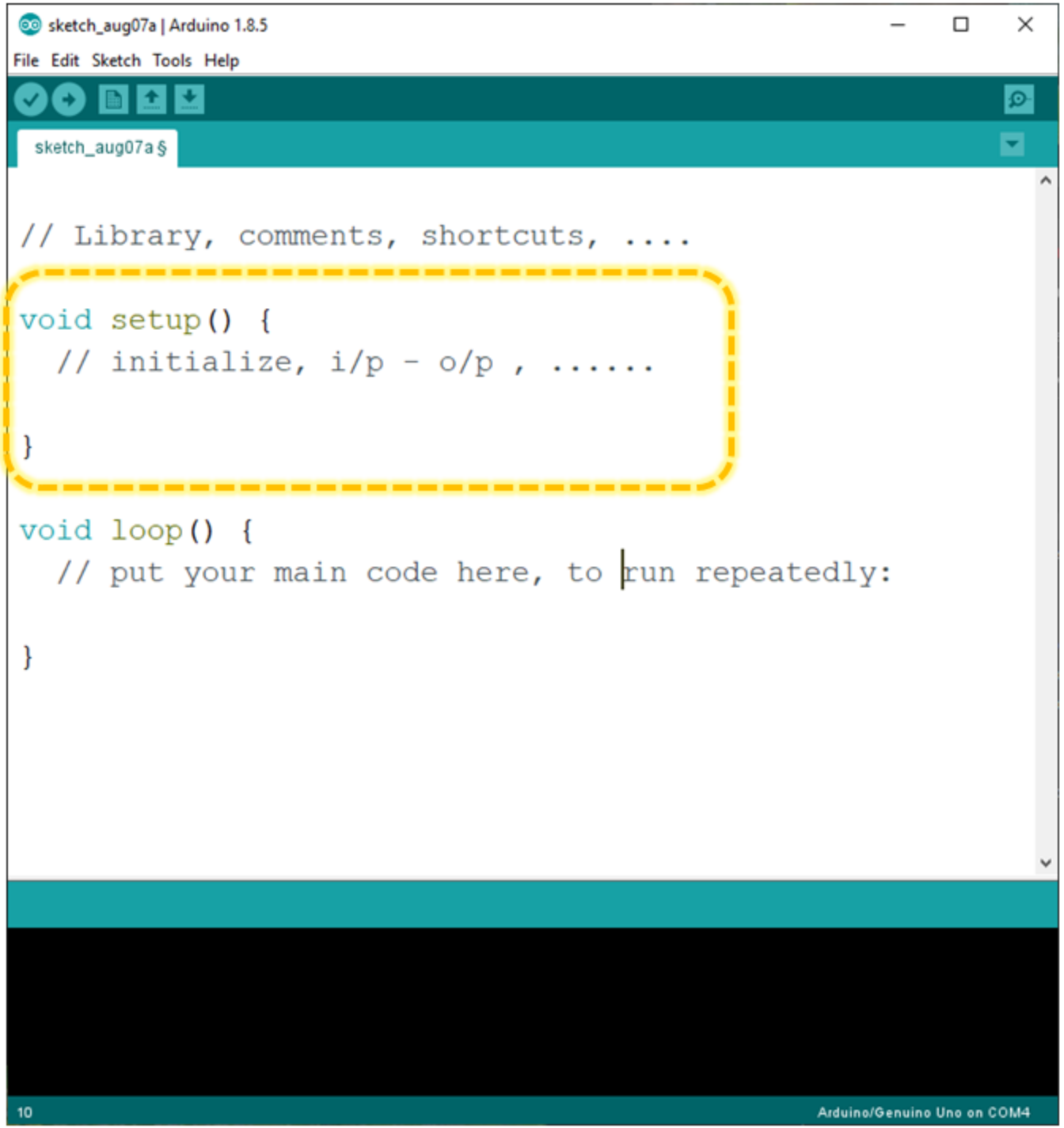
Windows Win 10 or newer (64-bit) DOWNLOAD

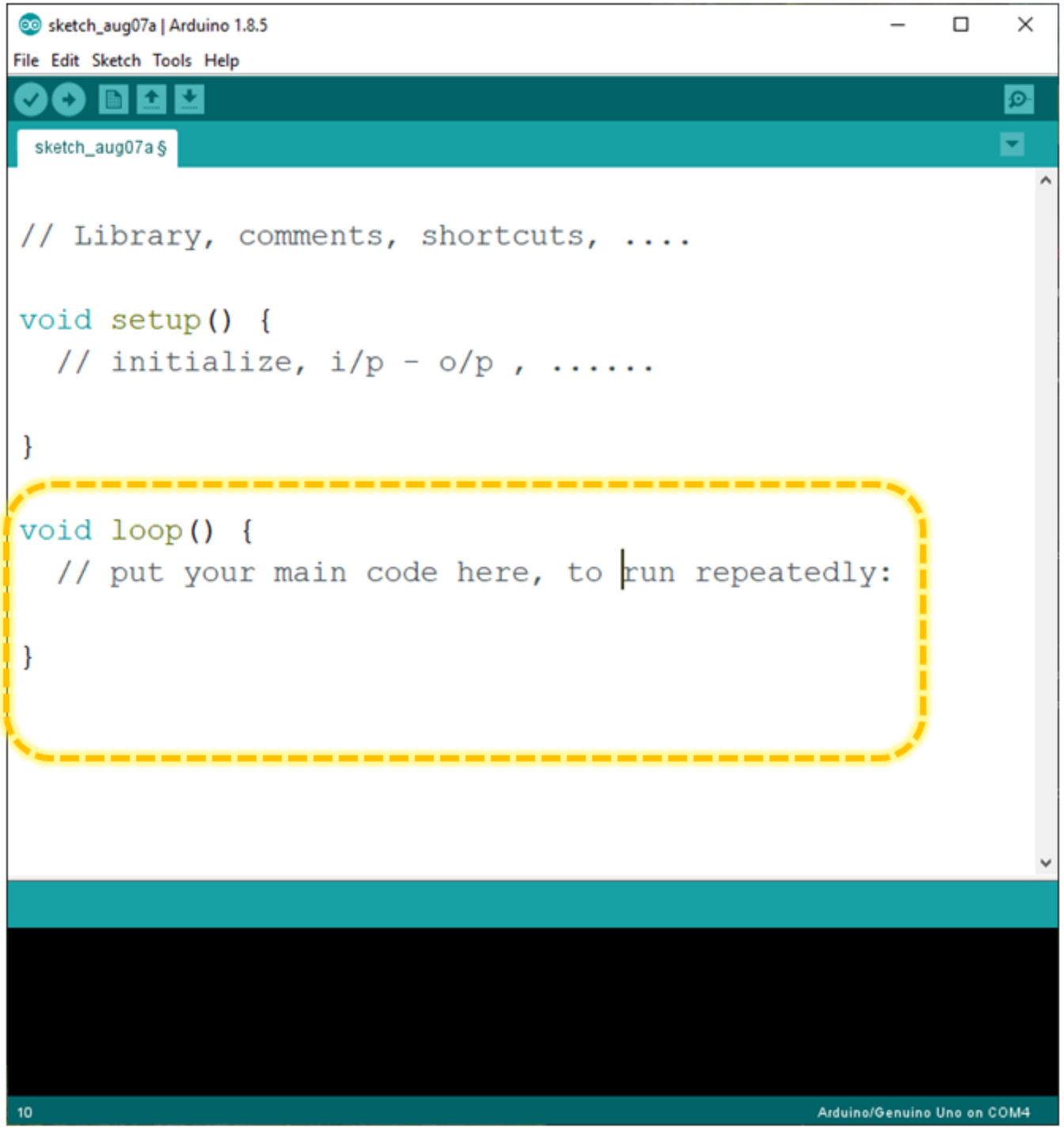
**Nightly Builds**

Download a preview of the incoming release with the most updated features and bugfixes.

The Arduino IDE 2.0 is open source and its source code is hosted on **GitHub**.

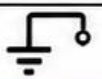

sketch_aug07a | Arduino 1.8.5
File Edit Sketch Tools Help
sketch_aug07a §
// Library, comments, shortcuts, ....
void setup() {
// initialize, i/p - o/p , ......
}
void loop() {
// put your main code here, to run repeatedly:
}
10
Arduino/Genuino Uno on COM4


sketch_aug07a | Arduino 1.8.5
File Edit Sketch Tools Help
sketch_aug07a §
// Library, comments, shortcuts, ....
void setup() {
// initialize, i/p - o/p , ......
}
void loop() {
// put your main code here, to run repeatedly:
}
10
Arduino/Genuino Uno on COM4


sketch_aug07a | Arduino 1.8.5
File Edit Sketch Tools Help
sketch_aug07a §
// Library, comments, shortcuts, ....
void setup() {
// initialize, i/p - o/p , ......
}
void loop() {
// put your main code here, to run repeatedly:
}
10
Arduino/Genuino Uno on COM4

# Essential Arduino Instructions

Common functions you will use to read inputs and control outputs

| Category | Instruction / Syntax | What it does (for students) | Example | Result / Illustration |
|---|---|---|---|---|
| 1. Setup & Program Structure | `void setup() { }` | Runs once when the Arduino starts. Used to set pin modes and initialize. | `void setup() { pinMode(13, OUTPUT); }` | Sets pin 13 as output after startup. |
| | `void loop() { }` | Runs repeatedly (forever). Contains the main code. | `void loop() { digitalWrite(13, HIGH); }` | Keeps running the code over and over. |
| | `pinMode(pin, mode)` | Sets a pin as INPUT or OUTPUT. | `pinMode(2, INPUT); pinMode(8, OUTPUT);` | Configures the pin behavior. |
| 2. Digital Input / Output | `digitalWrite(pin, value)` | Turns a digital pin ON (HIGH) or OFF (LOW). | `digitalWrite(8, HIGH); digitalWrite(8, LOW);` |  ON (HIGH)  OFF (LOW) |
| | `digitalRead(pin)` | Reads the state of a digital input pin (HIGH or LOW). | `int state = digitalRead(2);` |  Reads button state. |
| | `INPUT_PULLUP` | Enables the internal pull-up resistor (no external resistor needed). | `pinMode(2, INPUT_PULLUP);` |  Button connected to GND (active LOW). |
| 3. Analog Input | `analogRead(pin)` | Reads an analog value (0–1023) from sensors (e.g., light, temperature). | `int value = analogRead(A8);` |  Returns a value between 0 and 1023. |
| | `map(value, fromLow, fromHigh, toLow, toHigh)` | Converts a number from one range to another. | `int brightness = map(value, 0, 1023, 0, 255);` | Scales sensor value to 0–255. |
| | `analogReference(type)` | Sets the reference voltage for analog reading. | `analogReference(DEFAULT);` | Use default 5V reference (or INTERNAL / EXTERNAL). |
| 4. PWM Output (Analog Output) | `analogWrite(pin, value)` | Sends a PWM signal (0–255) to control brightness or speed. | `analogWrite(9, 128);` |  Controls LED brightness or motor speed. |
| 5. Time & Control Flow | `delay(ms)` | Pauses the program for a number of milliseconds. | `delay(1000);` |  Waits for 1 second. |
| | `if (condition) { }` | Runs code only if a condition is true. | `if (value > 500) { digitalWrite(8, HIGH); }` | Turns ON output if sensor value is high. |
| | `for (initialization; condition; update)` | Repeats code a specific number of times. | `for (int i = 0; i < 5; i++) { digitalWrite(8, HIGH); }` | Repeats the code 5 times. |
| 6. Useful Functions | `Serial.begin(baudRate)` | Starts serial communication with the computer. | `Serial.begin(9600);` |  Open Serial Monitor to see printed values. |
| | `Serial.println(value)` | Prints a value to the Serial Monitor. | `Serial.println(value);` | `Value: 512` `Temp: 23` Displays text and values to the Serial Monitor. |

# Arduino Programming: Words & Symbols

Read the symbol → Understand its meaning → Recognize it in code

| Word / Symbol | Meaning | Quick Example |
|---|---|---|
| HIGH / LOW | Digital logic levels: HIGH ≈ 5V, LOW ≈ 0V | `digitalWrite(13, HIGH);` |
| INPUT / OUTPUT | Pin modes: read a signal or send a signal. | `pinMode(2, INPUT);` |
| = vs == | = assigns a value; == compares two values. | `x = 5;  if (x == 5)` |
| ; | Marks the end of a C++ statement. | `int led = 13;` |
| { } | Group instructions inside a function or condition. | `if (x > 5) { ... }` |
| // | Begins a single-line comment; not executed. | `// Turn on the LED` |
| A0 | Name of an Arduino analog input pin | `analogRead(A0);` |

**Remember: Arduino uses C/C++-style syntax. Symbols change what your code means.**

# Project 1: Blink LED

## Purpose

- Blink one LED repeatedly.
- Learn digital output control and delay timing.
- The LED turns ON for 1 second, then OFF for 1 second, continuously.

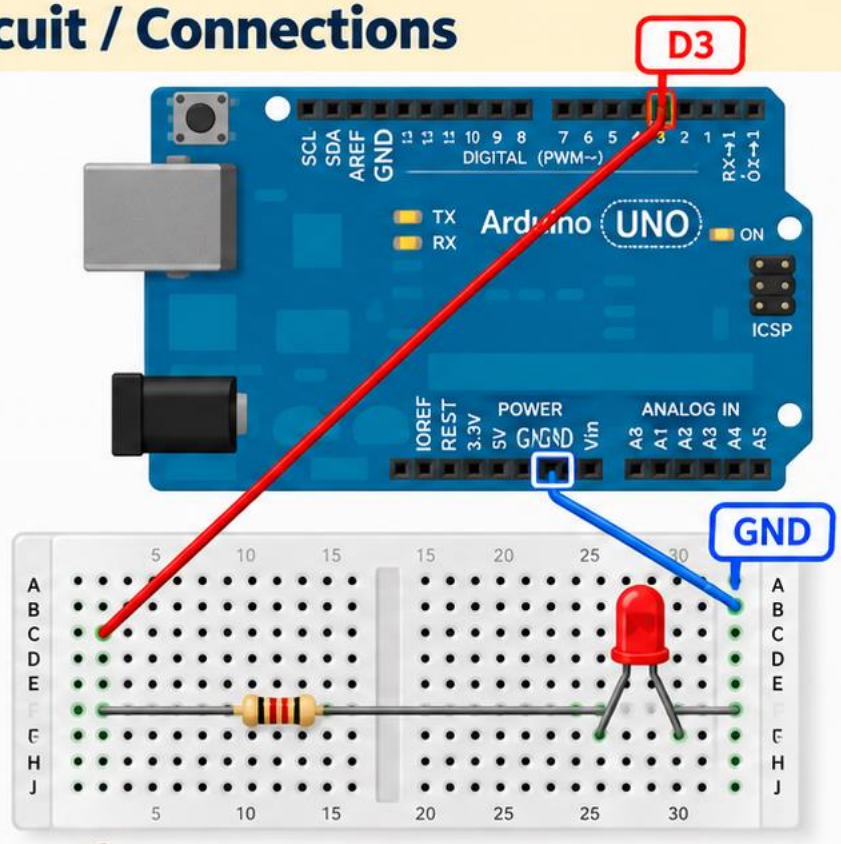
## Arduino Code

```
int LED = 3;

void setup() {
  pinMode(LED, OUTPUT);
}

void loop() {
  digitalWrite(LED, HIGH);
  delay(1000);
  digitalWrite(LED, LOW);
  delay(1000);
}
```

## Circuit / Connections




## Connection Summary

- D3 → 220Ω resistor → LED anode (+)
- LED cathode (−) → Arduino GND

# Project 2: Traffic Light System

## Purpose

- Simulate a simple traffic light using 3 LEDs.
- Learn digital output control and timing delays.
- The sequence is Green 2 s, Yellow 1 s, Red 2 s, then repeats.

## Arduino Code

```
int GreLED = 2;
int YelLED = 3;
int RedLED = 4;

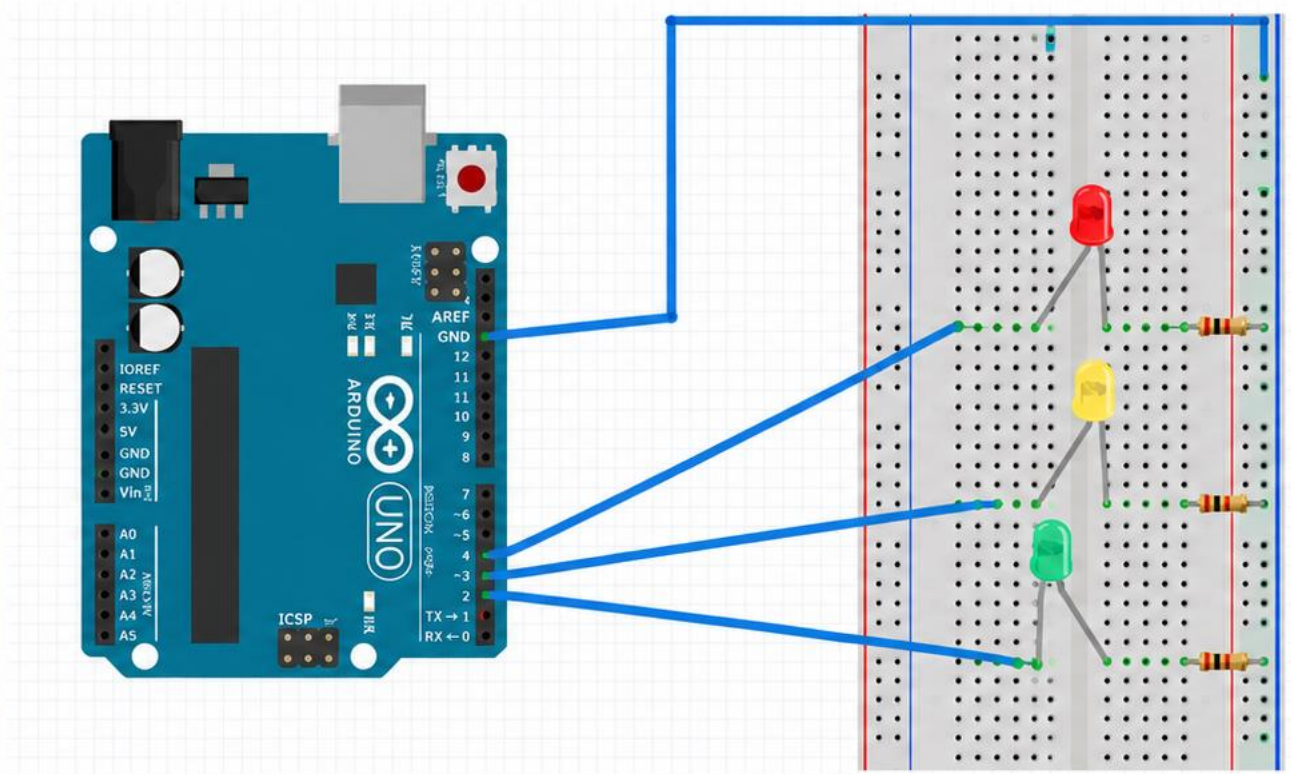
void setup() {
  pinMode(GreLED, OUTPUT);
  pinMode(YelLED, OUTPUT);
  pinMode(RedLED, OUTPUT);
}

void loop() {
  digitalWrite(GreLED, HIGH);
  digitalWrite(YelLED, LOW);
  digitalWrite(RedLED, LOW);
  delay(2000);

  digitalWrite(GreLED, LOW);
  digitalWrite(YelLED, HIGH);
  digitalWrite(RedLED, LOW);
  delay(1000);

  digitalWrite(GreLED, LOW);
  digitalWrite(YelLED, LOW);
  digitalWrite(RedLED, HIGH);
  delay(2000);
}
```

## Circuit / Connections (Original)




## Connection Summary

D2 → Green LED | D3 → Yellow LED | D4 → Red LED

- Each LED uses a 220 ohm resistor.
- All LED cathodes connect to GND.

# Project 3: Control LED Using Pushbutton

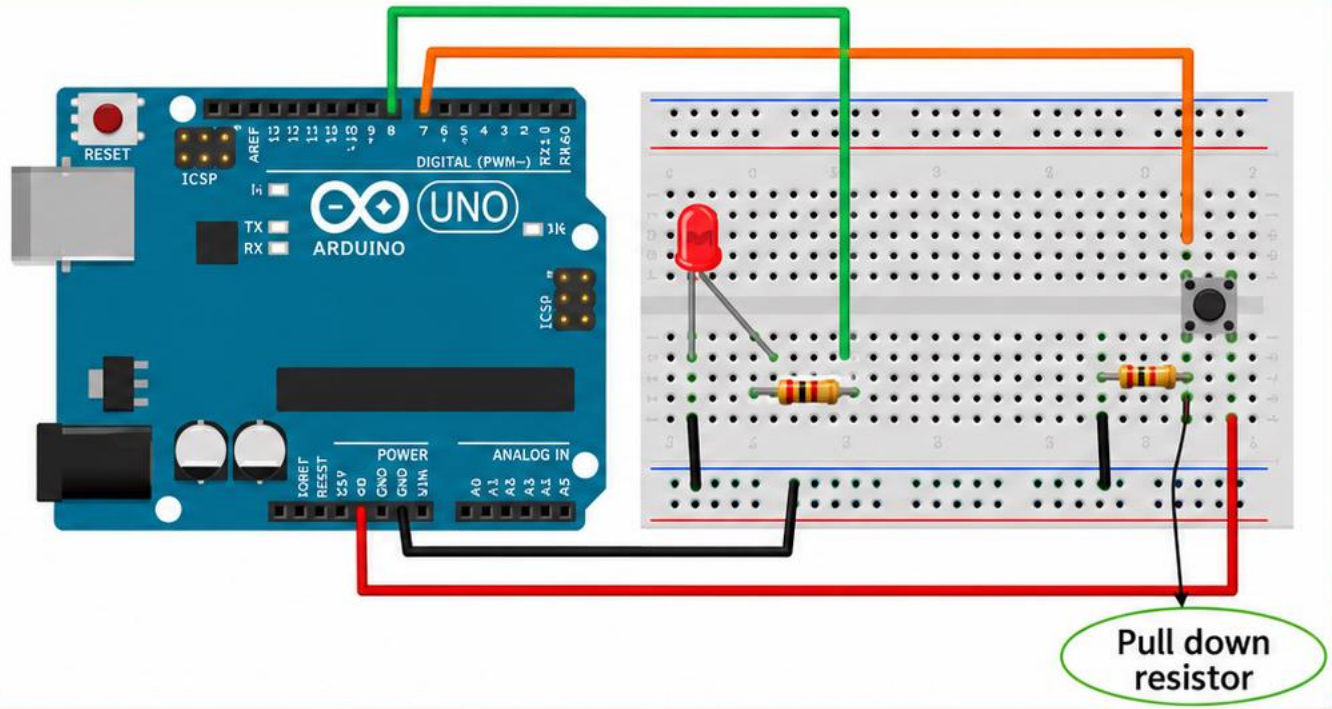
## Purpose

- Control an LED using a pushbutton.
- Learn digital input, digital output, and pull-down resistor usage.
- When the button is pressed, the LED turns ON; otherwise it stays OFF.

## Arduino Code

```
1  int LED = 12;
2  int BUTTON = 8;
3
4  void setup() {
5    pinMode(LED, OUTPUT);
6    pinMode(BUTTON, INPUT);
7  }
8
9  void loop() {
10   if (digitalRead(BUTTON) == HIGH) {
11     digitalWrite(LED, HIGH);
12   } else {
13     digitalWrite(LED, LOW);
14   }
15 }
```

## Circuit / Connections




## Connection Summary

- D12 → LED
- D8 → Pushbutton
- 220 Ω resistor → LED current limiting
- 10 kΩ resistor → Pull-down resistor
- Common GND used in the circuit

# Project 4: Alarm System

## Purpose

- Build a simple alarm system using a pushbutton, two LEDs, and a buzzer.
- Learn digital input, digital output, and pull-down resistor usage.
- If the pushbutton is OFF: Green LED ON, Red LED OFF, Buzzer OFF.
- If the pushbutton is ON: Green LED OFF, Red LED ON, Buzzer ON.

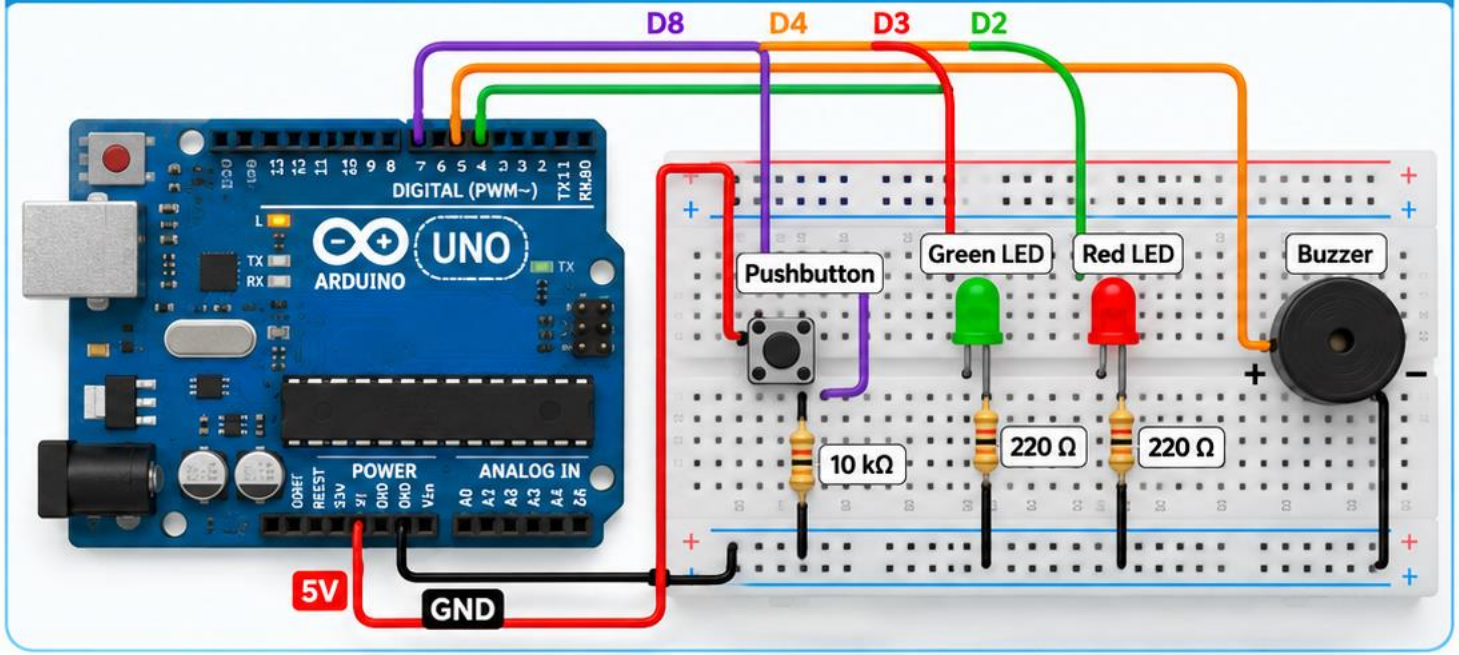
## Arduino Code

```
int BUTTON = 8;
int LEDG = 2;
int LEDR = 3;
int BUZZER = 4;

void setup() {
  pinMode(BUTTON, INPUT);
  pinMode(LEDG, OUTPUT);
  pinMode(LEDR, OUTPUT);
  pinMode(BUZZER, OUTPUT);
}

void loop() {
  if (digitalRead(BUTTON) == LOW) {
    digitalWrite(LEDG, HIGH);
    digitalWrite(LEDR, LOW);
    digitalWrite(BUZZER, LOW);
  } else {
    digitalWrite(LEDG, LOW);
    digitalWrite(LEDR, HIGH);
    digitalWrite(BUZZER, HIGH);
}
```

## Circuit / Connections




## Connection Summary

- D2 → Green LED (through 220 Ω resistor)
- D3 → Red LED (through 220 Ω resistor)
- D4 → Buzzer
- D8 → Pushbutton input
- 5V → Pushbutton supply
- 10 kΩ pull-down resistor → between D8 and GND
- All grounds → common GND

# Project 5: Potentiometer

## Purpose

- Read a variable analog voltage with Arduino.
- Use a potentiometer as a voltage divider.
- Display the sensor value in Serial Monitor.
- Learn analog input and ADC values (0–1023).

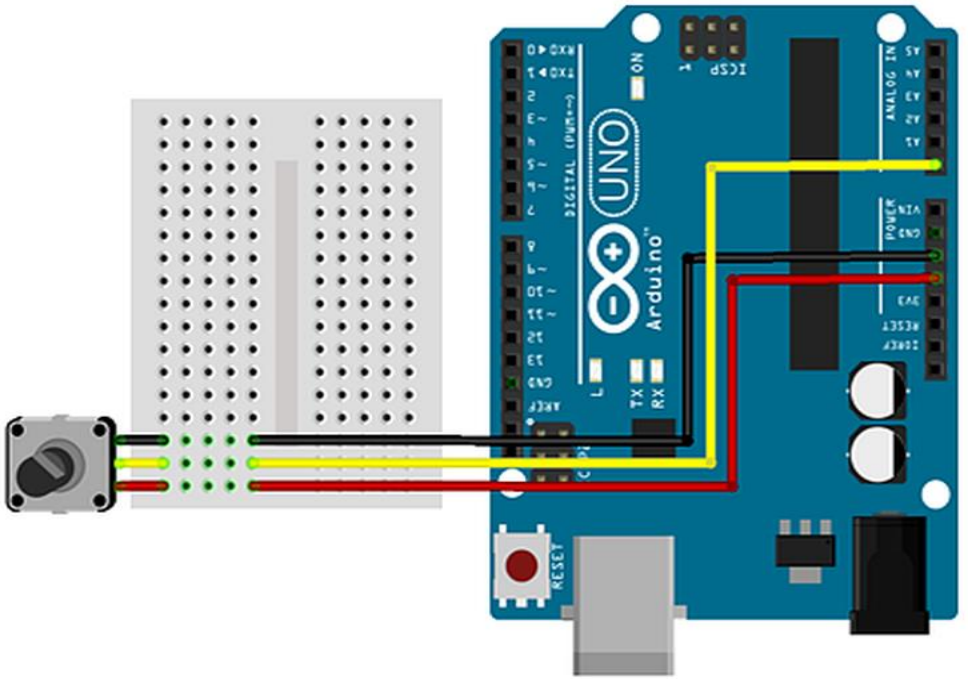
## Arduino Code

```
int poten = A0;
int sensor;

void setup() {
  Serial.begin(9600);
}

void loop() {
  sensor = analogRead(poten);
  Serial.println(sensor);
  delay(200);
}
```

## Circuit / Connections (Original)




## Connection Summary

Center terminal (wiper) → A0 (analog input)
Outer terminals → 5V and GND

## Analog-to-Digital Conversion (ADC)

Arduino UNO uses a 10-bit ADC: 2^10 = 1024 levels (readings 0–1023).

**ADC Value ≈ (Input Voltage / Reference Voltage) × 1023**

Example: (2.5 / 5) × 1023 ≈ 512

# Project 6: Control LEDs with Potentiometer

## Purpose

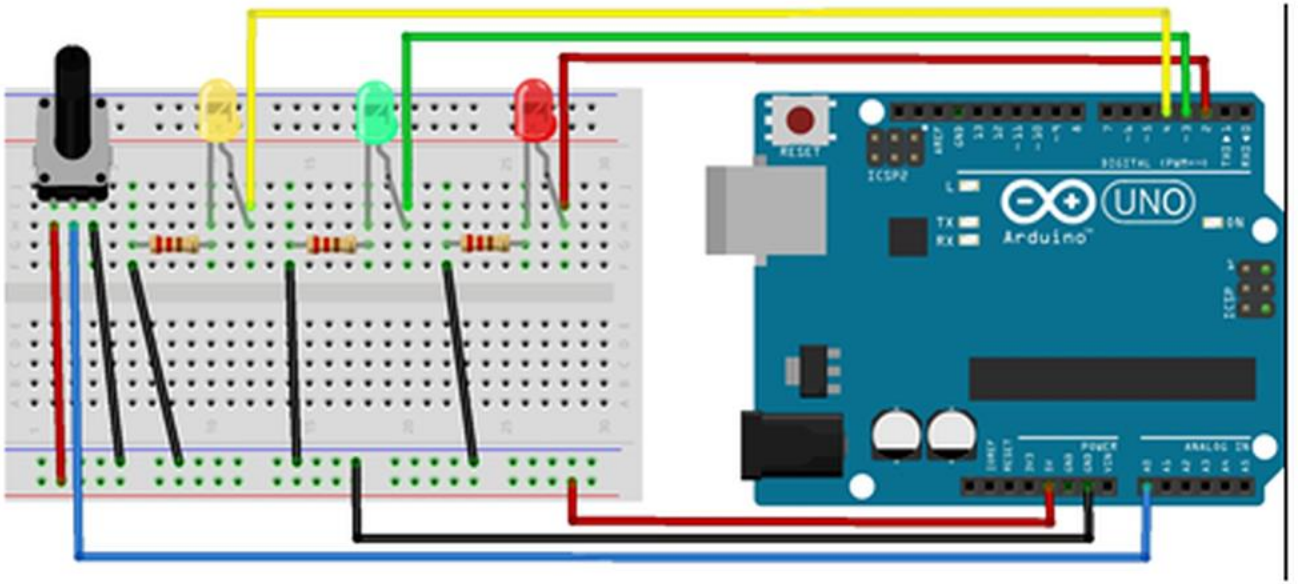
Use a potentiometer to control three LEDs.

Read A5 and select LED based on ADC value.

## Arduino Code

```
int GreLED = 2;
int YelLED = 3;
int RedLED = 4;
int poten = A5;
int sensor;  // missing declaration added
void setup() {
    pinMode(GreLED, OUTPUT);
    pinMode(YelLED, OUTPUT);
    pinMode(RedLED, OUTPUT);
    Serial.begin(9600);
}
void loop() {
    sensor = analogRead(poten);
    Serial.println(sensor);
    delay(200);
    if ((sensor >= 100) && (sensor < 500)) {
        digitalWrite(GreLED, HIGH);
        digitalWrite(YelLED, LOW);
        digitalWrite(RedLED, LOW);
        delay(2000);
    }
    else if ((sensor >= 500) && (sensor < 800)) {
        digitalWrite(GreLED, LOW);
        digitalWrite(YelLED, HIGH);
        digitalWrite(RedLED, LOW);
        delay(1000);
    }
    else if ((sensor >= 800) && (sensor < 1010)) {
        digitalWrite(GreLED, LOW);
        digitalWrite(YelLED, LOW);
        digitalWrite(RedLED, HIGH);
        delay(2000);
    }
    else {
        digitalWrite(GreLED, LOW);
        digitalWrite(YelLED, LOW);
        digitalWrite(RedLED, HIGH);
    }
}
```

## Circuit / Connections — Original




Original wiring preserved (no redrawing).

## Connection Summary

- **D2** → Green LED
- **D3** → Yellow LED
- **D4** → Red LED
- **A5** → Potentiometer wiper (center pin)

LEDs: use current-limiting resistors

Potentiometer outer pins: 5V / GND

# Project 7: LDR Light Sensor

## Purpose

- Detect ambient light using an LDR.
- Turn on the LED when it is dark.
- Read the sensor using analog pin A5.

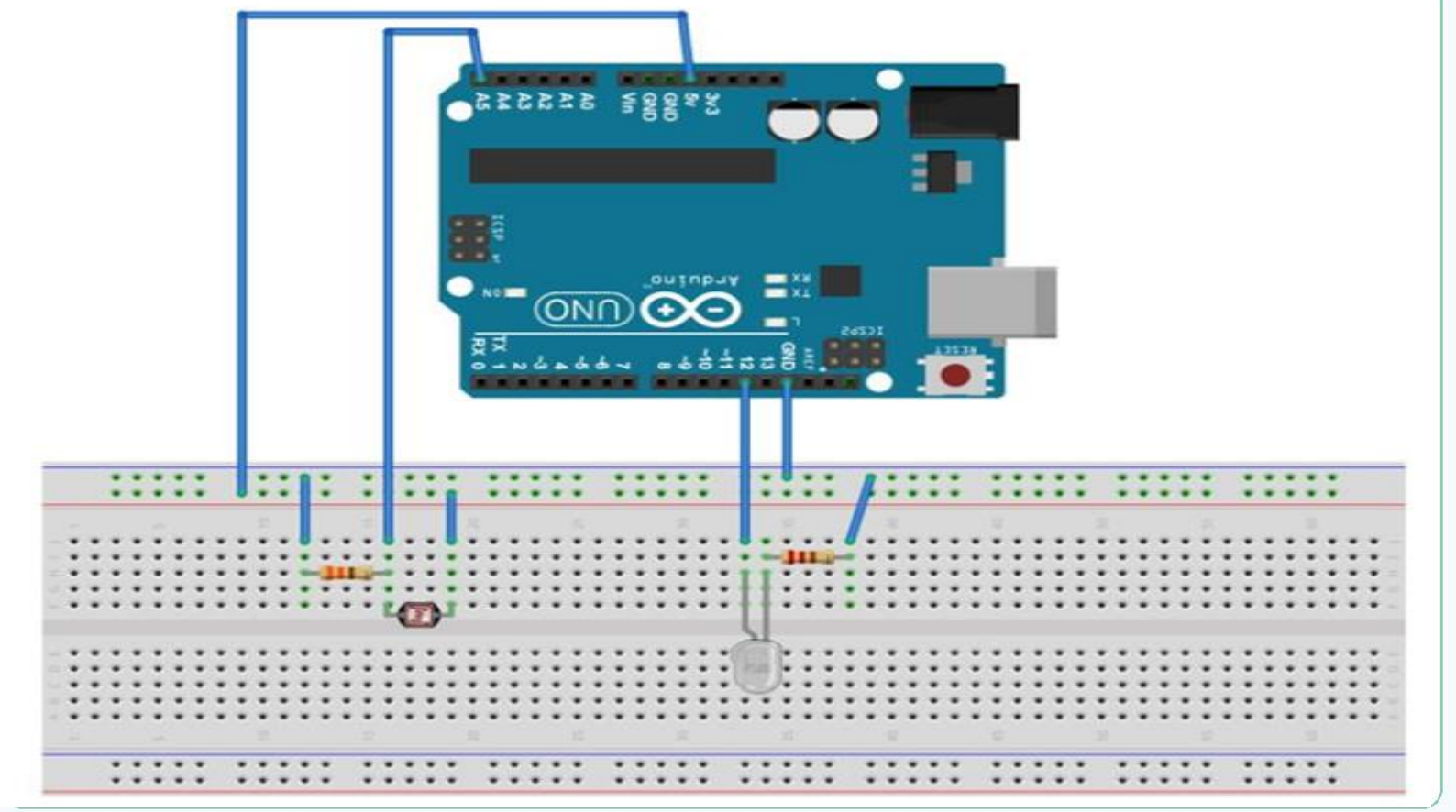
## Arduino Code

```
int ledPin = 12;
int ldrPin = A5;

void setup() {
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT);
  pinMode(ldrPin, INPUT);
}

void loop() {
  int ldrStatus = analogRead(ldrPin);
  if (ldrStatus <= 200) {
    digitalWrite(ledPin, HIGH);
    Serial.print("Its DARK, Turn on the LED : ");
    Serial.println(ldrStatus);
  } else {
    digitalWrite(ledPin, LOW);
    Serial.print("Its BRIGHT, Turn off the LED : ");
    Serial.println(ldrStatus);
  }
}
```

## Circuit / Connections — Original Wiring



## Connection Summary

**D12 → LED + 470 Ω resistor** **A5 → LDR–10 kΩ junction**

5V / GND → divider ends LED returns to GND